

Project 1: Data Preprocessing and Visualization

Course: DSC 101-Intro to Data Science

Gabriel Pereira

Libraries

The following codes import libraries necessary to run certain programs, applications, etc.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Research Questions

Do explicit songs have higher average track popularity than non-explicit songs?

Does song duration influence popularity?

Do artists with more followers tend to have higher track popularity?

Data Set

Source Citation:

Ward Abilal (2025). Spotify Global Music Dataset (2009–2025). Kaggle. Retrieved from <https://www.kaggle.com/datasets/wardabilal/spotify-global-music-dataset-20092025>

Dataset Background:

The dataset I chose to work with is Spotify Global Music Dataset (2009–2025), which was published on Kaggle by the user Warda Bilal. It contains streaming and other important information on thousands of songs released worldwide from 2009–2025. To build dataset, the creator collected data from the Spotify Web API. It includes details on track popularity, artist followers, genres, album release dates, and other key attributes used in music analytics.

This dataset is relevant to the field of music analysis, consumer behavior patterns on streaming services, and the entertainment industry as a whole because it demonstrates trends over time. Music streaming is a significant business model for artists, record labels, and streaming platforms. Thus, understanding factors that influence popularity such as track length, explicit content, and follower counts can help entertainment companies, record labels, and artists develop better marketing strategies to further improve relevancy in the music industry.

Dataset Loading

The dataset was loaded from two local CSV files. The first of which was spotify_data_clean.csv which includes data from modern (2025) songs. The second of which was track_data_final.csv which includes from classic (2009-2023) songs.

This code reads the modern Spotify dataset from a local CSV file and puts it into a pandas DataFrame named df_modern. The head() function is then used to display the first five rows to confirm that the data loaded correctly and to preview the structure of the dataset.

In [2]: df_modern = pd.read_csv('/Users/gabrielpereira/Desktop/dsc 101/spotify_data_clean.csv')
df_modern.head()

Out [2]:

	track_id	track_name	track_number	track_popularity	explicit
0	3EJS5LyekDim1Tf5rBFmZI	Trippy Mane (ft. Project Pat)	4	0	True
1	1oQW6G2ZiwMuHqIPpP27DB	OMG!	1	0	True
2	7mdkjzoiYlf1rx9EtBpGmU	Hard 2 Find	1	4	True
3	67rW0ZI7oB3qEpD5YWWE5w	Still Get Like That (ft. Project Pat & Starrah)	8	30	True
4	15xptTfRBrjsppW0INUZjf	ride me like a harley	2	0	True

This code reads the classic Spotify dataset from a local CSV file and puts it into a pandas DataFrame named df_classic. The head() function is then used to display the first five rows to confirm that the data loaded correctly and to preview the structure of the dataset.

```
In [3]: df_classic = pd.read_csv('/Users/gabrielpereira/Desktop/dsc 101/track_data_f
df_classic.head()
```

```
Out[3]:
```

	track_id	track_name	track_number	track_popularity	track_du
0	6pymOcrCnMuCWdgGVTvUgP		3	57	61
1	2IWc1iJlz2NVcStV5fbtPG	Clouds	1		67
2	1msEuwSBneBKpVCZQcFTsU	Forever & Always (Taylor's Version)	11		63
3	7bcy34fBT2ap1L4bfPsl9q	I Didn't Change My Number	2		72
4	0GLfodYacy3BJE7Al3A8en	Man Down	7		57

Inspecting the Size of the Modern Dataset

This code displays the number of observations (rows) and variables (columns) in the modern Spotify dataset.

```
In [4]: df_modern.shape
```

```
Out[4]: (8582, 15)
```

```
In [5]: df_classic.shape
```

```
Out[5]: (8778, 15)
```

Inspecting the Structure of the Dataset

This code displays the number of observations, column names, and data types of each variable in `df_modern` and `df_classic`. This helps verify the dataset structure and identify variable types (numerical, categorical, boolean).

```
In [6]: df_modern.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8582 entries, 0 to 8581
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   track_id              8582 non-null   object
1   track_name            8582 non-null   object
2   track_number          8582 non-null   int64
3   track_popularity      8582 non-null   int64
4   explicit              8582 non-null   bool
5   artist_name           8579 non-null   object
6   artist_popularity     8582 non-null   int64
7   artist_followers      8582 non-null   int64
8   artist_genres         5221 non-null   object
9   album_id              8582 non-null   object
10  album_name            8582 non-null   object
11  album_release_date    8582 non-null   object
12  album_total_tracks    8582 non-null   int64
13  album_type            8582 non-null   object
14  track_duration_min    8582 non-null   float64
dtypes: bool(1), float64(1), int64(5), object(8)
memory usage: 947.2+ KB
```

In [7]: `df_classic.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8778 entries, 0 to 8777
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   track_id              8778 non-null   object
1   track_name            8776 non-null   object
2   track_number          8778 non-null   int64
3   track_popularity      8778 non-null   int64
4   track_duration_ms     8778 non-null   int64
5   explicit              8778 non-null   bool
6   artist_name           8774 non-null   object
7   artist_popularity     8774 non-null   float64
8   artist_followers      8774 non-null   float64
9   artist_genres         8774 non-null   object
10  album_id              8778 non-null   object
11  album_name            8776 non-null   object
12  album_release_date    8778 non-null   object
13  album_total_tracks    8778 non-null   int64
14  album_type            8778 non-null   object
dtypes: bool(1), float64(2), int64(4), object(8)
memory usage: 968.8+ KB
```

Summary Statistics of Numerical Variables

This code generates descriptive statistics for all numerical variables in the `df_modern` and `df_classic`. This shows the mean, standard deviation, minimum, maximum, and quartiles. These statistics provide an overview of the distribution of key variables and can help identify potential outliers.

```
In [8]: df_modern.describe()
```

Out[8]:

	track_number	track_popularity	artist_popularity	artist_followers	album_total
count	8582.000000	8582.000000	8582.000000	8.582000e+03	8582.
mean	5.772547	52.356211	69.730016	2.403472e+07	13.
std	6.052792	23.816076	19.645979	3.803180e+07	11
min	1.000000	0.000000	0.000000	0.000000e+00	1.
25%	1.000000	39.000000	60.000000	4.623200e+05	6.
50%	4.000000	58.000000	74.000000	6.105547e+06	13.
75%	9.000000	71.000000	84.000000	2.725255e+07	17.
max	102.000000	99.000000	100.000000	1.455421e+08	181.

```
In [9]: df_classic.describe()
```

Out[9]:

	track_number	track_popularity	track_duration_ms	artist_popularity	artist_fo
count	8778.000000	8778.000000	8778.000000	8774.000000	8.77400
mean	5.749487	52.232855	210157.791866	69.954981	2.4354
std	6.017938	24.075221	63326.841454	19.532783	3.81370
min	1.000000	0.000000	0.000000	0.000000	0.00000
25%	1.000000	39.000000	173921.250000	60.000000	5.15276
50%	4.000000	58.000000	207276.000000	74.000000	6.27226
75%	9.000000	71.000000	239693.250000	84.000000	3.0550
max	102.000000	100.000000	811077.000000	100.000000	1.4554

Overview of Observations and Variables

The modern dataset contains a specific number of observations and variables. However, the classic dataset contains a similar structure with the same column names. Since both datasets share identical variable names and formats, they can be merged into a single combined dataset for further analysis.

The dataset includes:

Numerical variables (track_number, track_popularity, artist_popularity, artist_followers, album_total_tracks, track_duration_min)

Categorical variables (track_id, track_name ,artist_name ,artist_genres, album_id, album_name, album_type, era)

Boolean variables (explicit)

Variables of Interest

The primary variables analyzed in this project include:

track_popularity – Popularity score ranging from 0 to 100.

explicit – Indicates whether a track contains explicit content.

track_duration_min – Length of the track in minutes.

artist_followers – Number of followers the artist has on Spotify.

album_release_date – Date the album was released.

These variables will be used to analyze the relationships between song characteristics and popularity.

Data Cleaning and Preprocessing

Check for Missing Values

These code checks both datasets for missing values in each column to determine whether cleaning is necessary before merging.

```
In [10]: df_modern.isnull().sum()
```

```
Out[10]: track_id          0
track_name          0
track_number        0
track_popularity     0
explicit            0
artist_name         3
artist_popularity    0
artist_followers     0
artist_genres       3361
album_id            0
album_name          0
album_release_date   0
album_total_tracks   0
album_type          0
track_duration_min   0
dtype: int64
```

```
In [11]: df_classic.isnull().sum()
```

```
Out[11]: track_id      0
         track_name    2
         track_number  0
         track_popularity 0
         track_duration_ms 0
         explicit      0
         artist_name    4
         artist_popularity 4
         artist_followers 4
         artist_genres  4
         album_id      0
         album_name     2
         album_release_date 0
         album_total_tracks 0
         album_type     0
         dtype: int64
```

Handling Missing Values

A few of the missing values were found in artist_popularity, artist_followers, artist_genres, track_name, artists_genres, and album_name. Because the number of missing observations was very small in comparison to the total dataset size, these rows were removed. Dropping these rows prevents inaccurate statistical calculations while preserving the overall dataset integrity.

```
In [12]: df_modern = df_modern.dropna(subset=["artist_name", "track_name"])
         df_classic = df_classic.dropna(subset=["artist_name", "track_name", 'artist_
```

However, for df_modern, there were a significant amount of missing values in artists_genres. Thus, dropping these rows would have cause a significant loss in data, leading to innacurate findings. Rather than dropping them, I decided to fill these missing values with "Unknown" in order to preserve the data.

```
In [13]: df_modern["artist_genres"] = (df_modern["artist_genres"].replace("[]", np.na
```

Verifying Missing Values Were Removed

```
In [14]: df_modern.isnull().sum()
```

```
Out[14]: track_id      0
         track_name    0
         track_number  0
         track_popularity 0
         explicit      0
         artist_name   0
         artist_popularity 0
         artist_followers 0
         artist_genres  0
         album_id      0
         album_name     0
         album_release_date 0
         album_total_tracks 0
         album_type     0
         track_duration_min 0
         dtype: int64
```

```
In [15]: df_classic.isnull().sum()
```

```
Out[15]: track_id      0
         track_name    0
         track_number  0
         track_popularity 0
         track_duration_ms 0
         explicit      0
         artist_name   0
         artist_popularity 0
         artist_followers 0
         artist_genres  0
         album_id      0
         album_name     0
         album_release_date 0
         album_total_tracks 0
         album_type     0
         dtype: int64
```

Convert Duration Column to Ensure Consistency

The track_duration_ms column in the classic dataset was originally stored in milliseconds, while the modern dataset used minutes (track_duration_min). To ensure consistency before merging, the duration values in the classic dataset were converted from milliseconds to minutes. After conversion, the original millisecond column was removed to avoid redundancy.

```
In [16]: df_classic["track_duration_min"] = df_classic["track_duration_ms"] / 60000
```

```
In [17]: df_classic = df_classic.drop(columns=["track_duration_ms"])
```

Extract Release Year from Album Release Date

The album_release_date column contained inconsistent formats (some full dates and some year-only values). Instead of converting the column to full datetime format, the first four characters were extracted to get the release year. This ensures consistent numerical year values across both datasets.

```
In [18]: df_modern["release_year"] = df_modern["album_release_date"].astype(str).str[:4].astype(int)
df_classic["release_year"] = df_classic["album_release_date"].astype(str).str[:4].astype(int)
```

Drop the Original Release Date Column

Since the analysis only requires the release year, the original album_release_date column is dropped to simplify the dataset and avoid duplications and repetitiveness.

```
In [19]: df_modern = df_modern.drop(columns=["album_release_date"])
df_classic = df_classic.drop(columns=["album_release_date"])
```

Create Release Decade Feature

Grouped the data by decade to simplify analysis of trends.

```
In [20]: df_modern["release_decade"] = (df_modern["release_year"] // 10) * 10
df_classic["release_decade"] = (df_classic["release_year"] // 10) * 10
```

Create Duration Category Feature

To simplify analysis, track duration was grouped into three categories based on length (Short, Medium, and Long). This feature helps analyze whether song length influences popularity.

```
In [21]: df_modern["duration_category"] = pd.cut(
    df_modern["track_duration_min"],
    bins=[0, 2.5, 4, df_modern["track_duration_min"].max()],
    labels=["Short", "Medium", "Long"])

df_classic["duration_category"] = pd.cut(
    df_classic["track_duration_min"],
    bins=[0, 2.5, 4, df_classic["track_duration_min"].max()],
    labels=["Short", "Medium", "Long"])
```

Since both datasets now share identical column names and consistent units, they were merged vertically into a single combined dataset for analysis.

```
In [22]: combined_df = pd.concat([df_modern, df_classic], ignore_index=True)
```

Verify Merge

This confirms the size of the combined dataset after merging.

```
In [23]: combined_df.shape
```

```
Out[23]: (17351, 17)
```

Create Popularity Level Category Feature

In order to simplify analysis, track popularity was grouped into three categories: Low, Medium, and High. This allows for easier comparison across groups.

```
In [24]: combined_df["popularity_level"] = pd.cut(
    combined_df["track_popularity"],
    bins=[0, 40, 70, 100],
    labels=["Low", "Medium", "High"]
)
```

Merge the Modern and Classic Datasets

Convert Selected Variables to Categorical Type

Certain variables were converted to categorical type to improve grouping efficiency and visualization clarity.

```
In [25]: combined_df["explicit"] = combined_df["explicit"].astype("category")
combined_df["album_type"] = combined_df["album_type"].astype("category")
combined_df["popularity_level"] = combined_df["popularity_level"].astype("category")
combined_df["duration_category"] = combined_df["duration_category"].astype("category")
```

Data Visualization

Research Question 1

Do explicit songs have higher average track popularity than non-explicit songs?

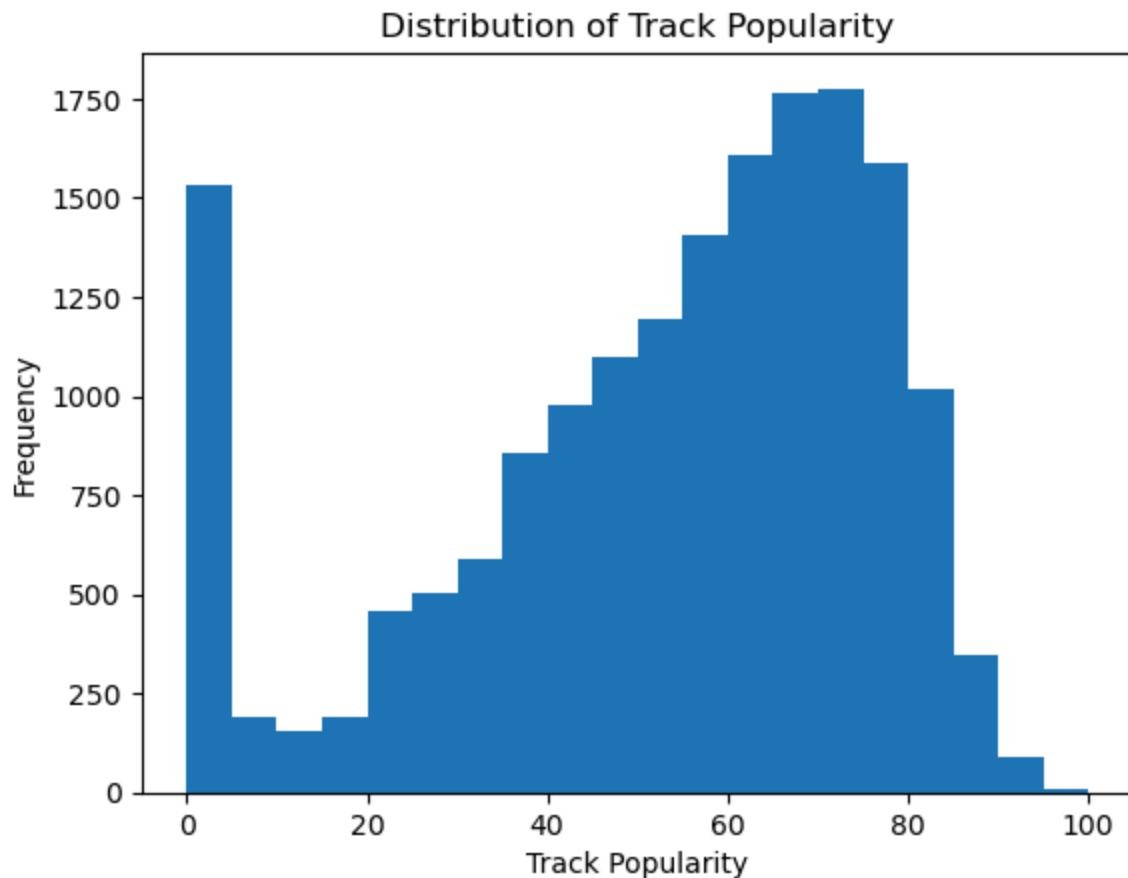
Histogram of Track Popularity

This histogram shows the overall distribution of track popularity scores. It helps identify the shape, center, and spread of the popularity variable.

```
In [26]: plt.hist(combined_df["track_popularity"], bins=20)

plt.title("Distribution of Track Popularity")
```

```
plt.xlabel("Track Popularity")  
plt.ylabel("Frequency")  
plt.show()
```



Explanation

The histogram shows that track popularity is moderately left-skewed, meaning that most songs have mid to high popularity scores (around 50-80). The center of the distribution appears around 60-70. There is also a noticeable spike at 0 popularity, implying there is a group of songs with very low or no measurable popularity. This may represent newly released tracks or songs with minimal streaming activity. The overall spread covers a range from 0-100 on the track popularity scale.

Average Popularity by Explicit Status

This code calculates the average track popularity for explicit and non-explicit songs

```
In [27]: explicit_means = combined_df.groupby("explicit", observed=True)["track_popularity"].mean()  
explicit_means
```

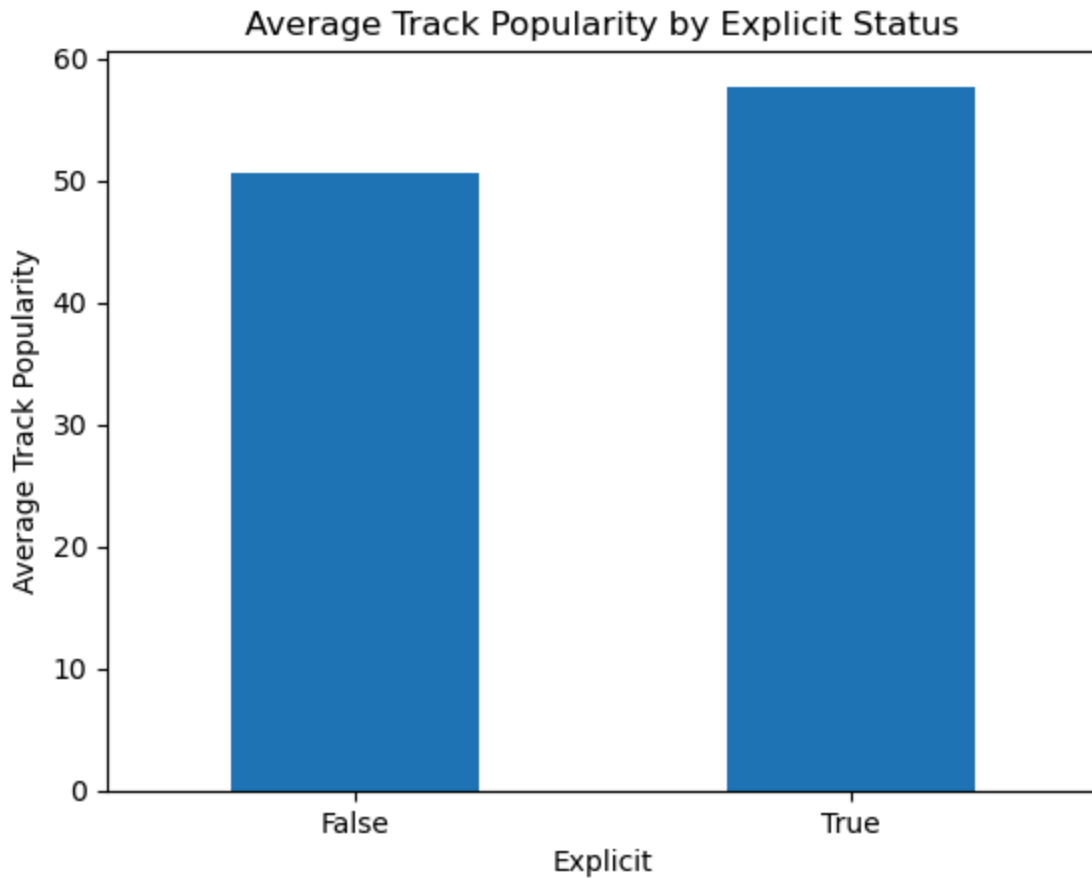
```
Out[27]: explicit  
False    50.507607  
True     57.673507  
Name: track_popularity, dtype: float64
```

Bar Char Showing Explicit vs Non-Explicit Popularity

This bar chart visualizes the average popularity for explicit and non-explicit tracks.

```
In [28]: explicit_means.plot(kind="bar")

plt.title("Average Track Popularity by Explicit Status")
plt.xlabel("Explicit")
plt.ylabel("Average Track Popularity")
plt.xticks(rotation=0)
plt.show()
```



Explanation:

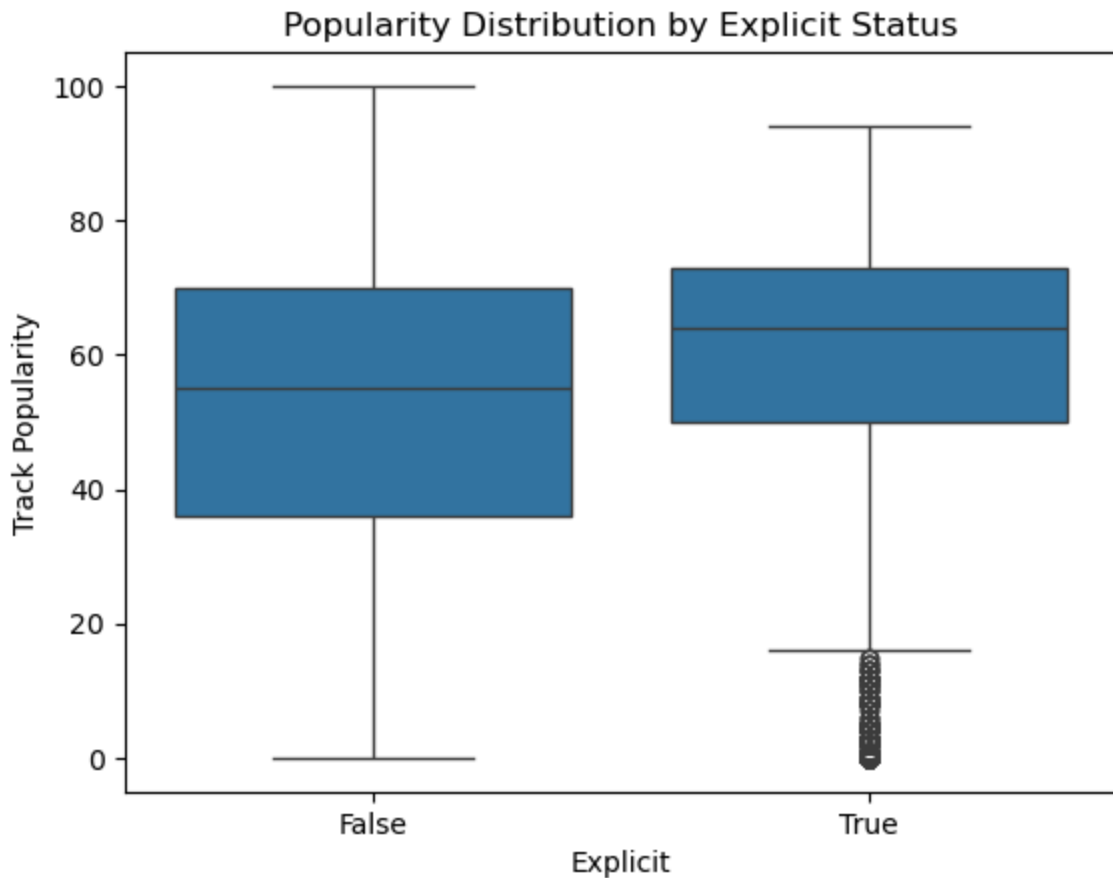
The bar chart shows that explicit songs have a higher average track popularity than non-explicit songs. On average, explicit tracks score approximately 7–8 popularity points higher. This suggests that explicit content may be associated with slightly greater listener engagement and appeal.

Boxplot Showing Popularity by Explicit Status

This boxplot compares the distribution, median, and spread of popularity for explicit and non-explicit songs.

```
In [29]: sns.boxplot(x="explicit", y="track_popularity", data=combined_df)

plt.title("Popularity Distribution by Explicit Status")
plt.xlabel("Explicit")
plt.ylabel("Track Popularity")
plt.show()
```



Explanation:

The boxplot shows that explicit songs have a higher median popularity than non-explicit songs. The interquartile range for explicit tracks is shifted upward, indicating that the middle 50% of explicit songs tend to have higher popularity scores. Both groups show variability across the full 0–100 range, but explicit songs appear to have more low-popularity outliers. Overall, the distribution supports the earlier finding that explicit tracks tend to have higher average popularity.

Research Question 2

Does song duration influence popularity?

Popularity by Duration Category

This code calculates the average popularity for each duration category.

```
In [30]: duration_means = combined_df.groupby("duration_category", observed=True)["track_popularity"].mean()
```

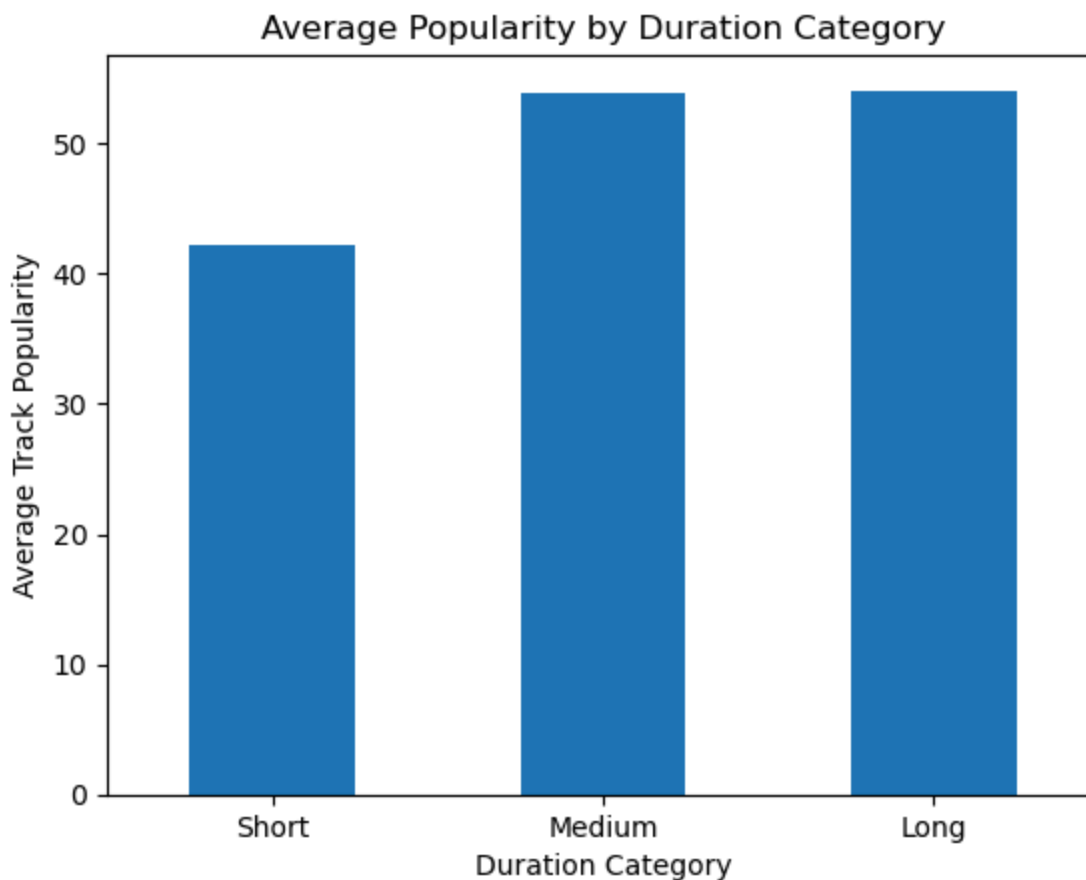
```
Out[30]: duration_category
Short      42.245148
Medium     53.815910
Long       54.073978
Name: track_popularity, dtype: float64
```

Bar Chart: Average Popularity by Duration Category

This bar chart compares the average track popularity across Short, Medium, and Long duration categories to determine whether song length influences popularity.

```
In [31]: duration_means.plot(kind="bar")

plt.title("Average Popularity by Duration Category")
plt.xlabel("Duration Category")
plt.ylabel("Average Track Popularity")
plt.xticks(rotation=0)
plt.show()
```



Explanation:

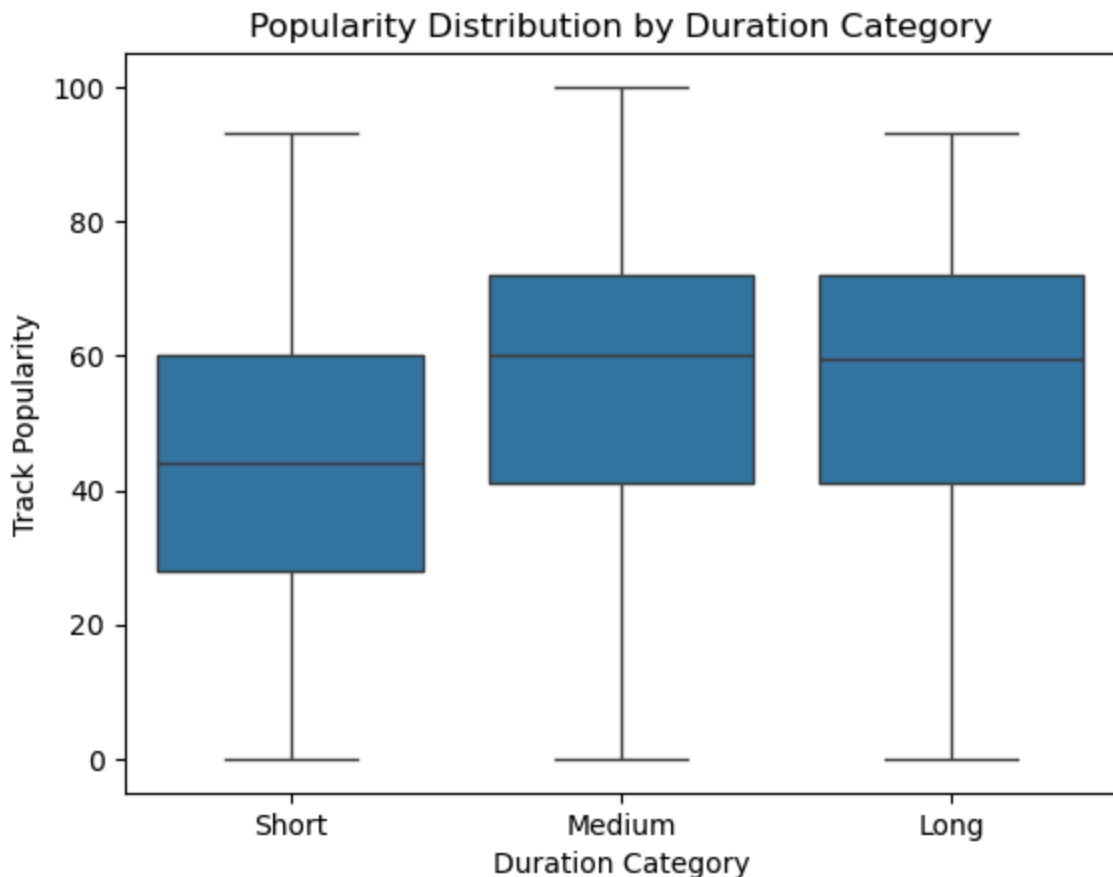
The bar chart shows that Short songs have an average popularity of approximately 42, while Medium and Long songs have higher average popularity scores of about 54–55. This indicates that Medium and Long tracks are, on average, about 12–13 points more popular than Short tracks. The similarity between Medium and Long categories suggests that once a song exceeds a short duration, additional length does not significantly increase popularity.

Boxplot Showing Popularity by Duration Category

This boxplot compares the distribution, median, and variability of track popularity across Short, Medium, and Long duration categories.

```
In [32]: sns.boxplot(x="duration_category", y="track_popularity", data=combined_df)

plt.title("Popularity Distribution by Duration Category")
plt.xlabel("Duration Category")
plt.ylabel("Track Popularity")
plt.show()
```



Explanation

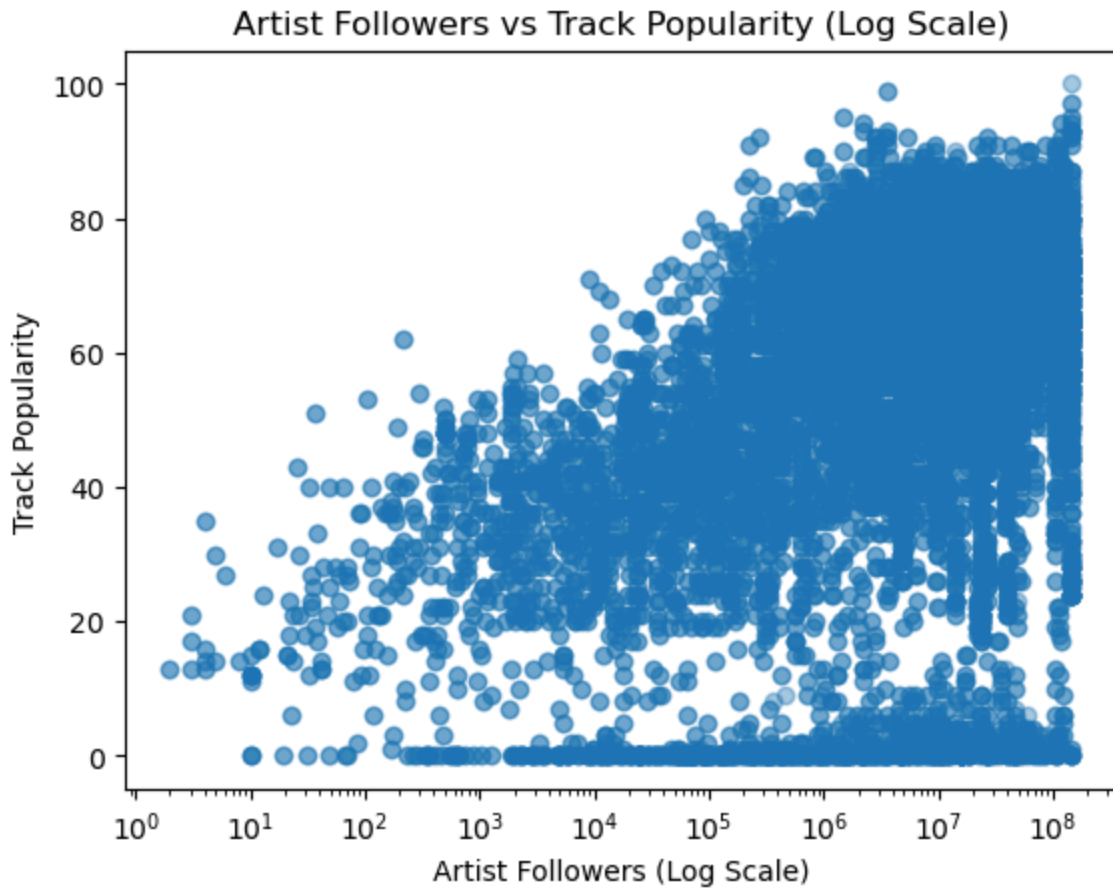
The boxplot shows that Short songs have a median popularity of approximately 45, while Medium and Long songs have higher medians of about 60. This aligns with the bar chart results, where Medium and Long tracks averaged around 54–55, compared to 42 for Short tracks. The interquartile ranges for Medium and Long songs are similar and shifted upward compared to Short songs, indicating that the middle 50% of Medium and Long tracks tend to perform better. Overall, the distribution suggests that very short songs are generally less popular, while Medium and Long songs show comparable popularity levels.

Research Question 3

Do artists with more followers tend to have higher track popularity?

Scatter Plot: Followers vs Popularity

```
In [33]: plt.scatter(  
    combined_df["artist_followers"],  
    combined_df["track_popularity"],  
    alpha=0.4  
)  
  
plt.xscale("log")  
  
plt.title("Artist Followers vs Track Popularity (Log Scale)")  
plt.xlabel("Artist Followers (Log Scale)")  
plt.ylabel("Track Popularity")  
plt.show()
```



Explanation:

The scatter plot uses a logarithmic scale for artist followers to account for the wide range of follower counts. The plot shows a positive relationship between artist followers and track popularity, as popularity generally increases with follower size. However, the relationship appears moderate rather than strong, since there is substantial variability in popularity at all follower levels. This suggests that while larger follower bases are associated with higher popularity, follower count alone does not fully determine a song's success.

Correlation Calculation

This calculates the correlation coefficient between artist followers and track popularity.

```
In [34]: correlation = combined_df["artist_followers"].corr(combined_df["track_popularity"])
          correlation
```

```
Out[34]: np.float64(0.2291831791851084)
```

Explanation:

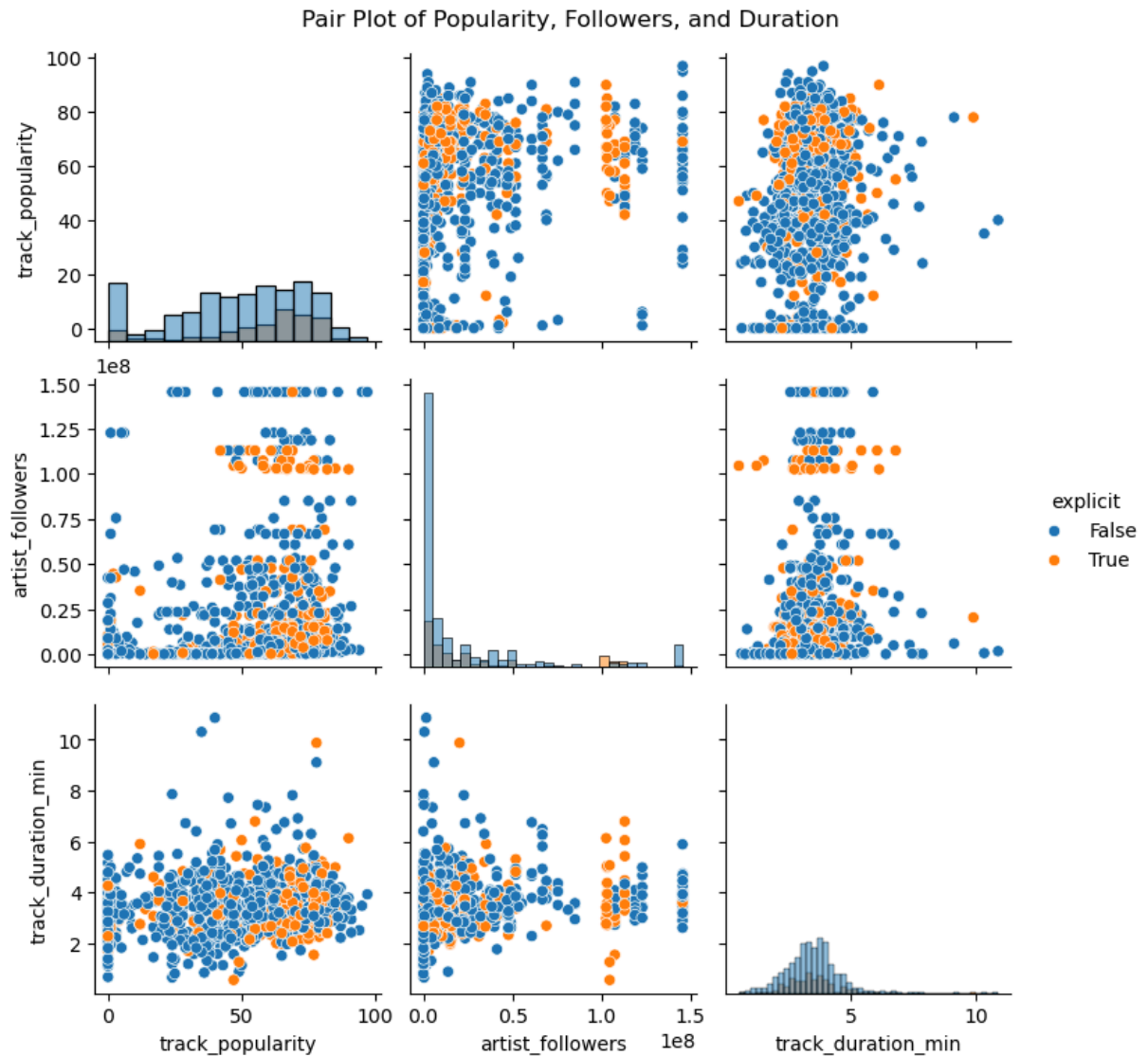
The correlation coefficient between artist followers and track popularity is approximately 0.23, indicating a weak positive relationship. This suggests that artists with more followers tend to have slightly higher track popularity, but follower count alone does not strongly predict a song's popularity. Although the relationship is positive, the low correlation value indicates that many other factors likely influence track popularity beyond follower count.

Pair Plot of Key Numerical Variables

```
In [35]: sample_df = combined_df[[
            "track_popularity",
            "artist_followers",
            "track_duration_min",
            "explicit"
        ]].dropna().sample(1000, random_state=42)

sns.pairplot(
    sample_df,
    hue="explicit",
    diag_kind="hist"
)

plt.suptitle("Pair Plot of Popularity, Followers, and Duration", y=1.02)
plt.show()
```



Explanation:

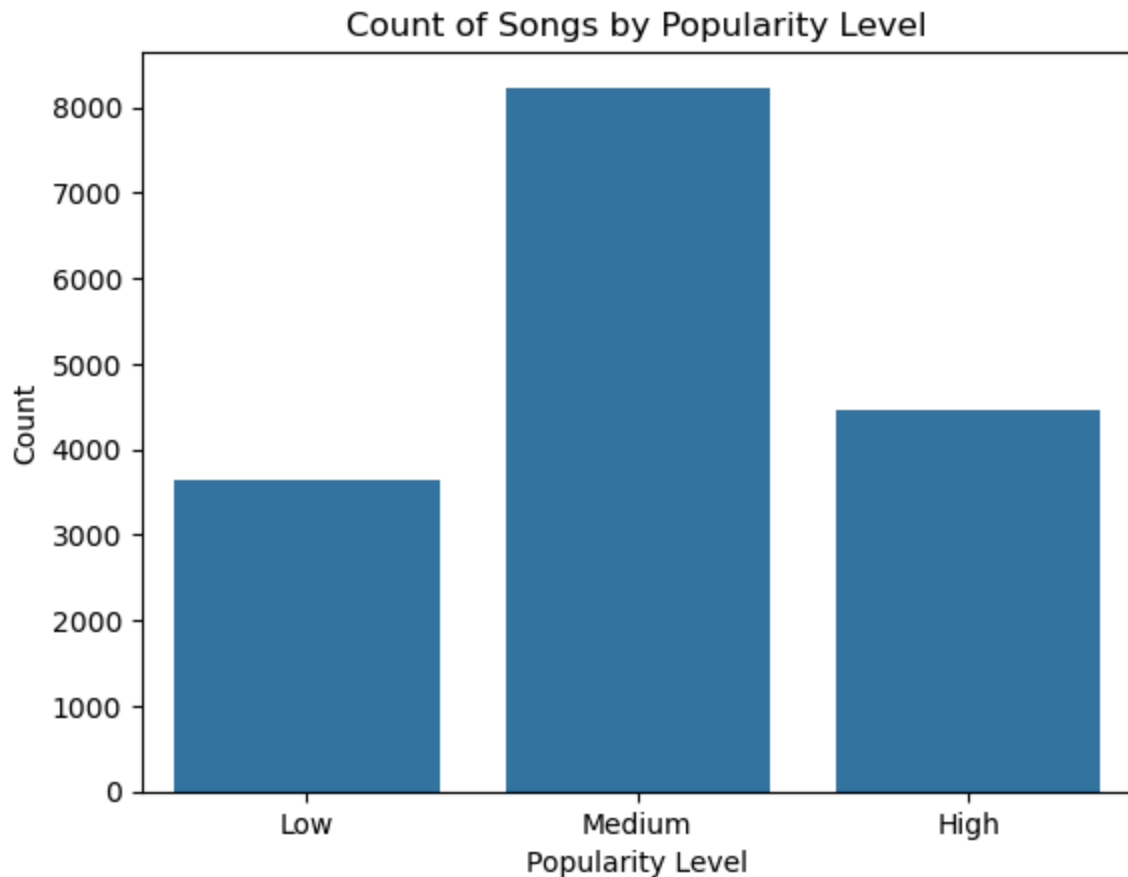
The pairplot provides a comprehensive view of relationships among track popularity, artist followers, and track duration. The diagonal histograms confirm that popularity is left-skewed with a concentration in the mid-to-high range, while artist followers are highly right-skewed. The scatter plots show a weak positive relationship between followers and popularity, consistent with the correlation coefficient ($r \approx 0.23$). In contrast, the relationship between duration and popularity appears weak, with no strong visible pattern. Overall, the pairplot reinforces earlier findings that follower count has some influence on popularity, while duration has a limited effect.

Categorical Plot

This categorical plot shows the count of songs in each popularity level category.

```
In [36]: sns.countplot(x="popularity_level", data=combined_df)

plt.title("Count of Songs by Popularity Level")
plt.xlabel("Popularity Level")
plt.ylabel("Count")
plt.show()
```



Explanation:

This bar chart shows the number of songs classified into Low, Medium, and High popularity categories. The majority of songs fall into the Medium category (approximately 8,200 songs), followed by the High category (around 4,500 songs), while the Low category contains the fewest songs (approximately 3,600). This distribution suggests that most tracks achieve moderate popularity levels, with fewer songs performing at either extreme. The concentration in the Medium category aligns with the histogram results, where most popularity scores were clustered in the mid-range.

Summary and Findings

This project analyzed Spotify track data to investigate the factors that influence a songs popularity. I was interested in this dataset because music has been something I have always loved since I was a kid. I've grown up playing instruments, infact I can play the

piano, trumpet, and accordion. Music has always been something I wanted to have a career in, and being able to combine data analysis with music was absolutely fascinating. Additionally, I found this interesting because music streaming plays a major role in modern culture. Understanding what drives popularity is very important for artists, producers, and streaming platforms. Spotify popularity scores reflect listener engagement, therefore making this dataset helpful in finding patterns audience behavior.

My first research question I investigate was whether or not explicit songs have higher popularity than non-explicit songs. The results indicate that explicit tracks actually have a higher average popularity, scoring around 7-8 points higher than non-explicit tracks. The boxplot produced confirmed this finding as it showed a higher median popularity and an upward shifted interquartile range for explicit songs. Both categories showed variability across the full 0-100 popularity range, but there was a consistent upward shift which suggests that explicit content may be associated with greater listener engagement and appeal.

The second research question I investigated was whether or not song duration influences popularity. The results showed that short songs, which had an average popularity of around 42/100, performed noticeable worse than medium and long songs, which had an average popularity of around 54-55/100. This difference in popularity, around 12-13 points, suggests that short songs may be less engaging to listeners, in turn decreasing their streaming and popularity. However, the similarity between the medium and long song categories popularity indicates that once a song exceeds a short duration, additional length to the song duration does not significantly increase popularity. The boxplot produced further confirmed that medium and long songs have similar medians of around 60/100 popularity, while short songs have a lower median of around 45/100 popularity. Overall, song duration seems to only have a significant impact on song popularity when the songs are unusually short.

The third research question I investigated was whether artists with more followers tend to have higher track popularity or not. The scatter plot produced revealed that there was a positive relationship between artist followers and popularity. However, the correlation coefficient, which was around 0.23, indicates that this relationship is weak. Even though artists with larger follower bases generally do achieve higher popularity, follower count does not solely predict success. There is significant variability in popularity across all the follower levels, suggesting that other factors like marketing, playlist placement, genre trends, or exposure through social media like Tik Tok can play a significant role.

The histogram produced that shows track popularity revealed there was a left-skewed distribution, where most of the songs were concentrated in the 50-80 range with a significant spike at 0 popularity. This spike may represent newly released songs that have minimal streaming activity. The overall spread of the histogram covers a full range of 0-100, indicating there is high variability in performance across tracks.

From a business perspective, these findings valuable for Spotify and music industry stakeholders. Being aware that explicit songs and medium to long duration songs tend to perform better can help guide marketing strategies, playlist creation, and artist development. Additionally, understanding that follower count only has a weak prediction in popularity suggests that platforms should not solely rely on artist size when promoting tracks. Growing and emerging artists may still achieve high popularity through strong marketing, promotion, or even viral exposure.

Furthermore, this project also connects to mindfulness by encouraging data backed and driven thinking rather than simply assumption based conclusions. For example, it may be much easier to assume that longer songs or highly followed artists automatically perform better, but the data shows that the relationships may not be as significant as previously thought. Being mindful when analyzing data involves carefully observing patterns, interpreting evidence objectively rather than subjectively, and recognizing limitations.

If I were to continue this project, the next step I would take would be to build a predictive model through machine learning techniques or other means in order to estimate track popularity based on multiple variables simultaneously. This would allow for a deeper understanding of how different factors interact to influence success.

Overall, the analysis of this dataset was able to sufficiently answer all of my research questions and revealed meaningful information on how explicit content, duration, and follower count relate to Spotify track popularity.

In []: